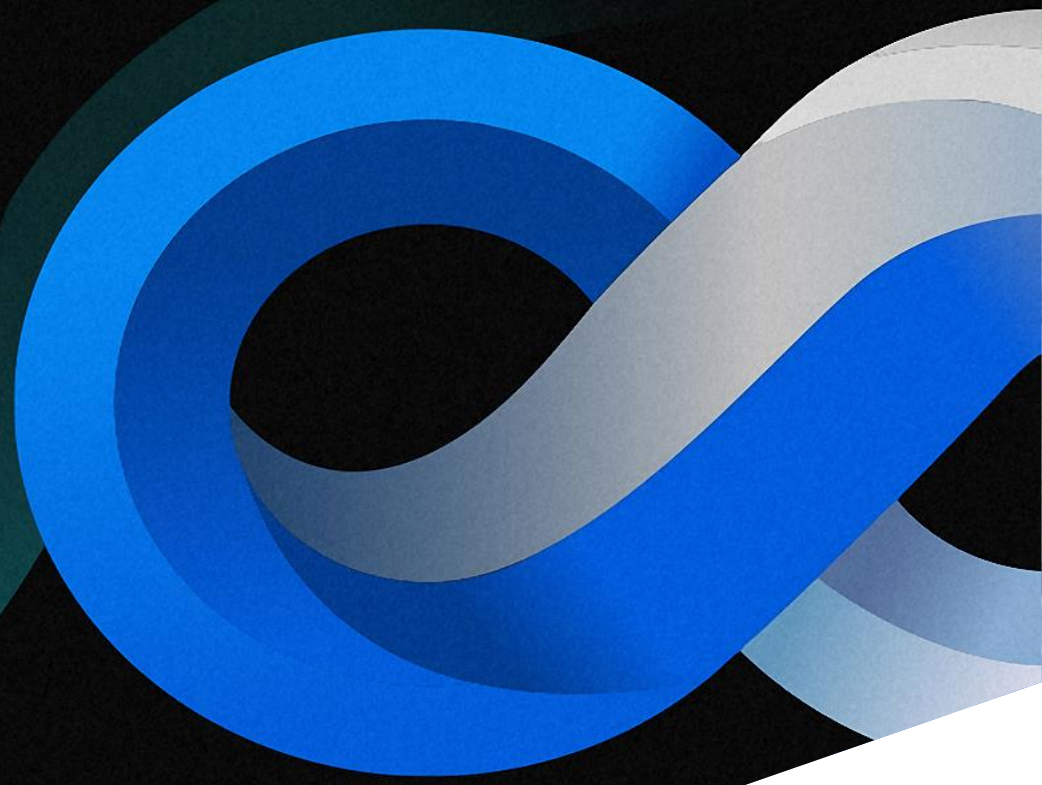
BBF-412-1 PYTHON PROGRAMMING TRAINING



Agile Testing Coach Team

BMW BRILLIANCE
AUTOMOTIVE

BACKGROUND

According to AWM is the Feature Team responsible of the generated quality of the product (including execution of functional / non-functional tests, according to specifications). Below is the Qualification Map Testing, it will help you to find the right training course, points out the mandatory training courses and shows the available training courses per competence area.

1 2 3

The recommended sequence is to start with session 1. For session 2-3, you can participate in the training according to the needs of the project

Agile Testing Methodology

[Session 1] Basic Knowledge of Agile Testing Process
[Session 2] Functional Test Case Design Method
[Session 3] Defect Management in Xray
[Session 4] Test Data Preparation Basic Method - TBD

The recommended sequence is to start with python related training (Session 1 and 2) then choose the related topic based on project demands(Session 3, 4 or 5)

Auto-Test

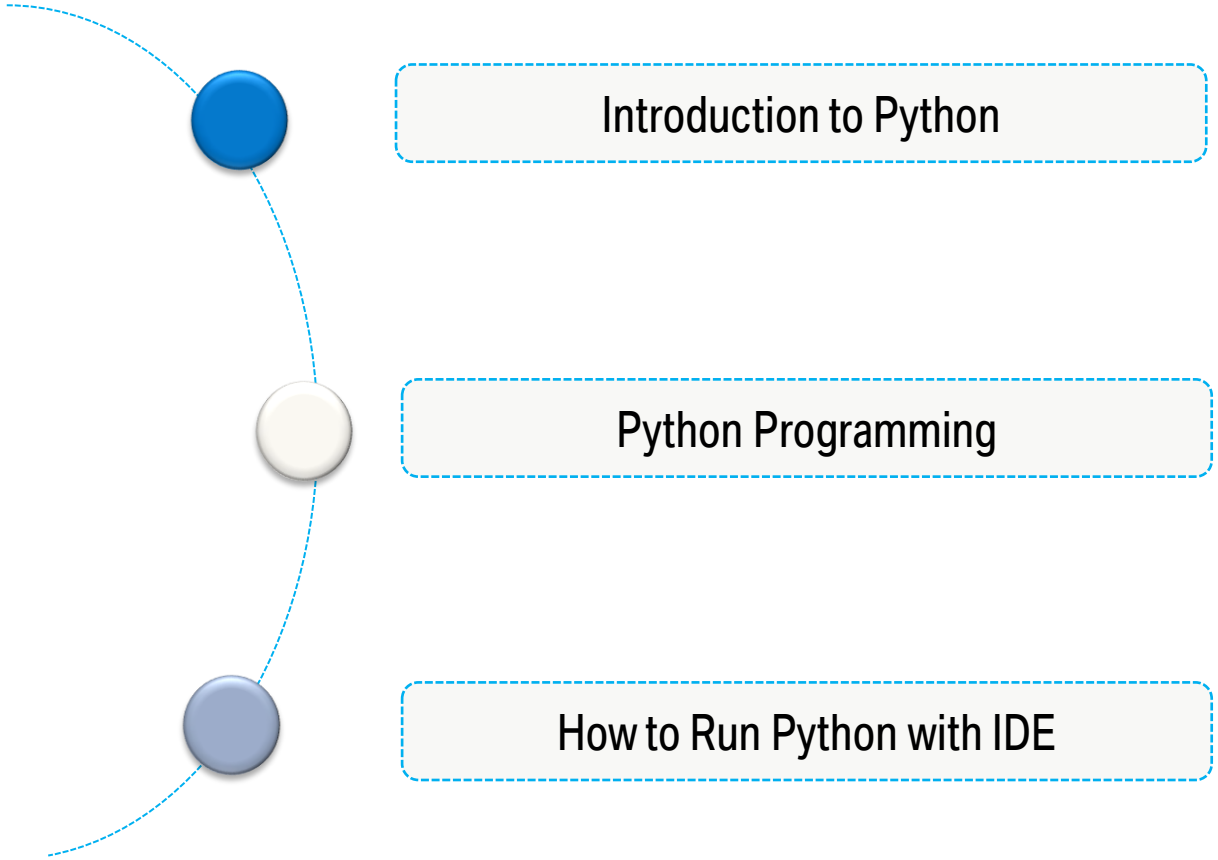
[Session 1] Python Basic Training for Testing
[Session 2] Basic Knowledge of Pytest-BDD Test Framework
[Session 3] Web Automation Testing with AATF Python
[Session 4] Mobile Automation Testing with AATF Python
[Session 5] API Automation Testing with AATF Python - TBD

It is strongly recommended to complete the training in the order of session (from Session 1 to 4), and attend other related trainings based on project/tester demands

Performance Test

[Session 1] Performance - Basic Knowledge
[Session 2] Performance Test Plan & Design
[Session 3] Performance Test Tools & Scripts
[Session 4] Performance Test Execution & Report

AGENDA


Introduction to Python
Python Programming
How to Run Python with IDE

Part 1: What is Python

THE STORY OF PYTHON

404

page not found.

MerryChristmas

Santa Claus works hard!

In December 1989, a Dutch programmer called **Guido van Rossum** was looking for a "hobby" project to keep him occupied over his Christmas holiday. He decided to call his project "Python" after the famous British comedy troupe, "Monty Python's Flying Circus."

Van Rossum goes on to explain:

It all started with ABC, a wonderful teaching language that I had helped create in the early eighties. It was an incredibly elegant and powerful language, aimed at non-professional programmers. Despite all its elegance and power and the availability of a free implementation, ABC never became popular in the Unix/C world. I decided not to repeat this mistake in Python.

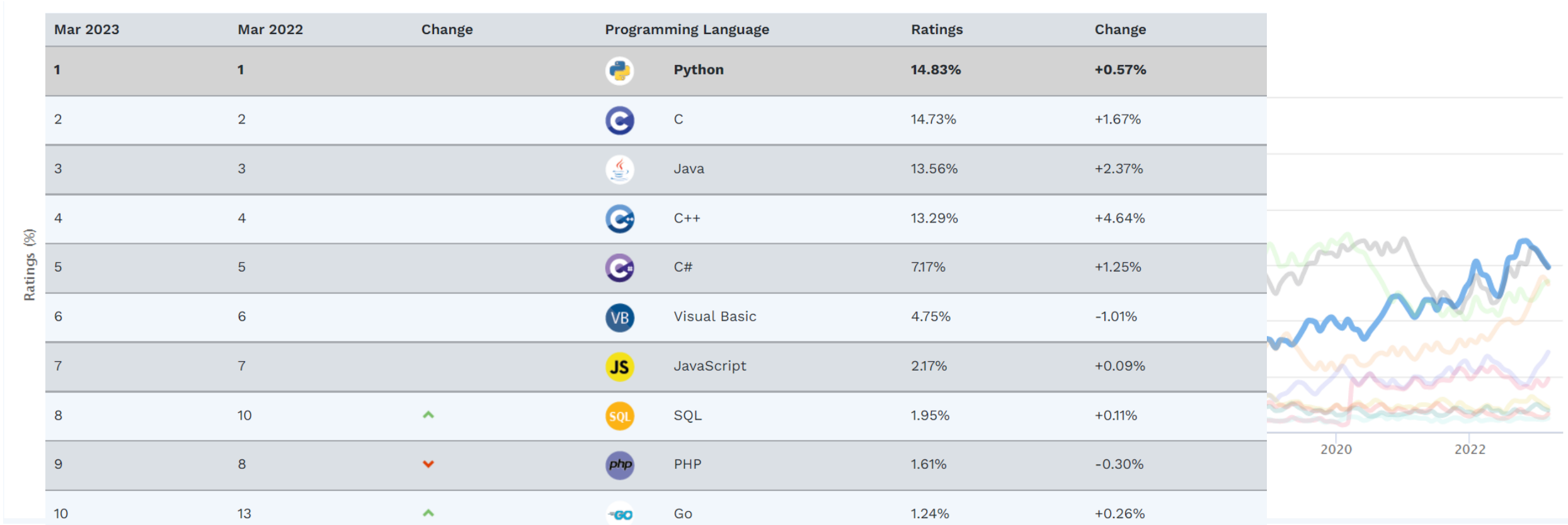
Perhaps this explains why Python is so popular in education: **from the beginning**, it was derived from a language **designed for teaching** and aimed at **nonprofessional programmers**.

WHY WE CHOOSE PYTHON

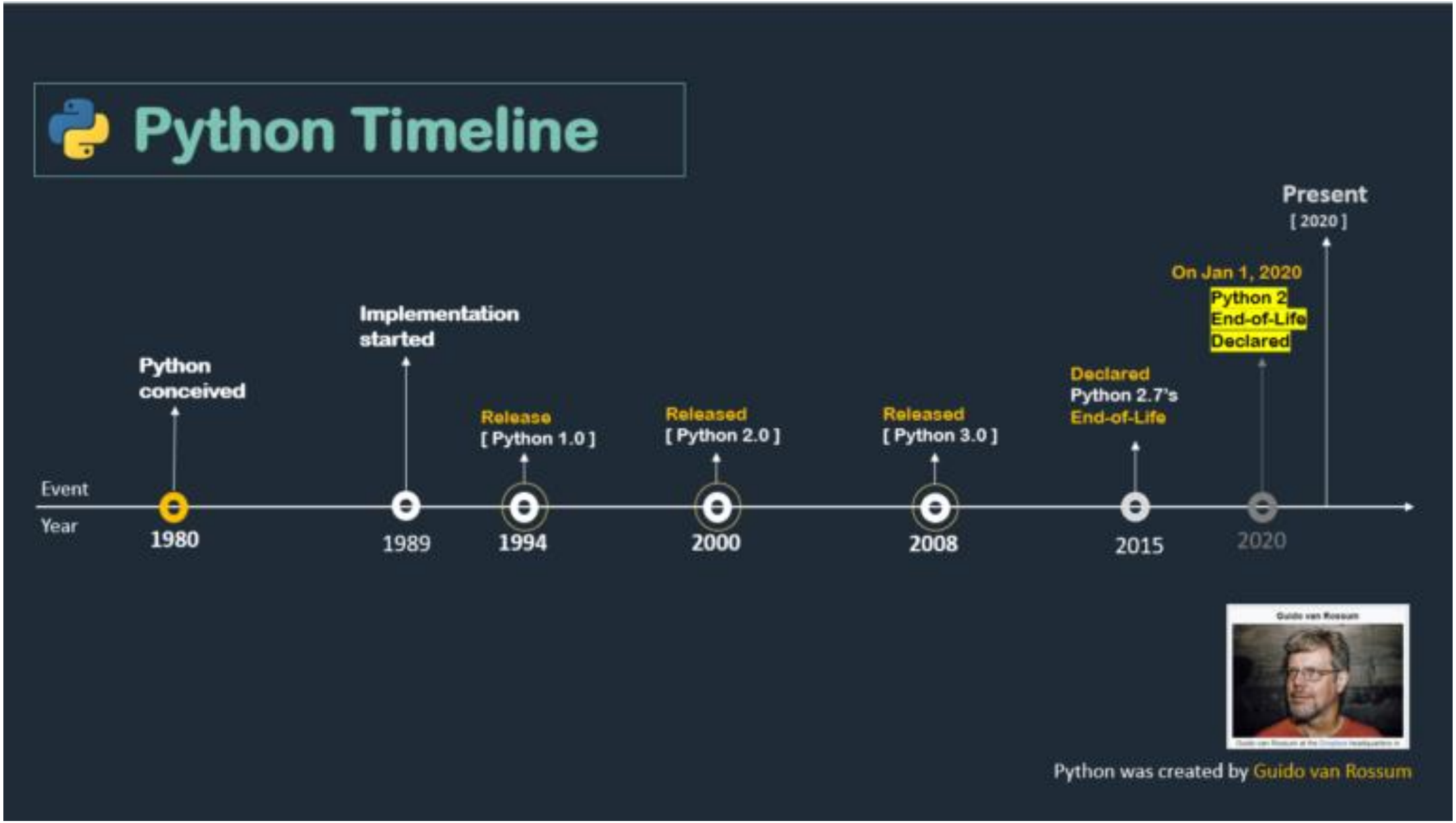
Python is powerful... and fast;
plays well with others;
runs everywhere;
is friendly & easy to learn;
is Open.

IN PYTHON… YOU'LL NEVER WALK ALONE

Mar 2023	Mar 2022	Change	Programming Language	Ratings	Change
1	**1**		**Python**	**14.83%**	**+0.57%**
2	2		C	14.73%	+1.67%
3	3		Java	13.56%	+2.37%
4	4		C++	13.29%	+4.64%
5	5		C#	7.17%	+1.25%
6	6		Visual Basic	4.75%	-1.01%
7	7		JavaScript	2.17%	+0.09%
8	10	▲	SQL	1.95%	+0.11%
9	8	▼	PHP	1.61%	-0.30%
10	13	▲	Go	1.24%	+0.26%



PYTHON TIMELINE

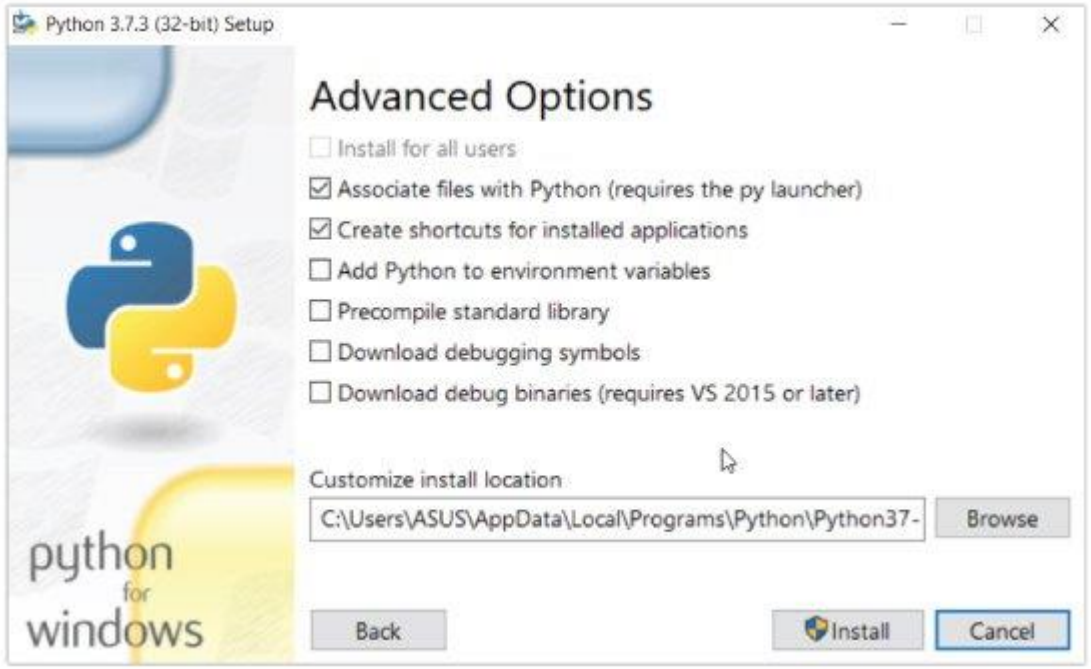


Part 2: How to use Python

INSTALL PYTHON SEPARATELY

1.Download the latest version of Python.
2.Run the installer file and follow the steps to install Python
During the install process, check **Add Python to environment variables**. This will add Python to environment variables, and you can run Python from any part of the computer.

- Even though most of today's Linux and Mac have Python pre-installed in it, the version might be out-of-date. So, it is always a good idea to install the most current version.

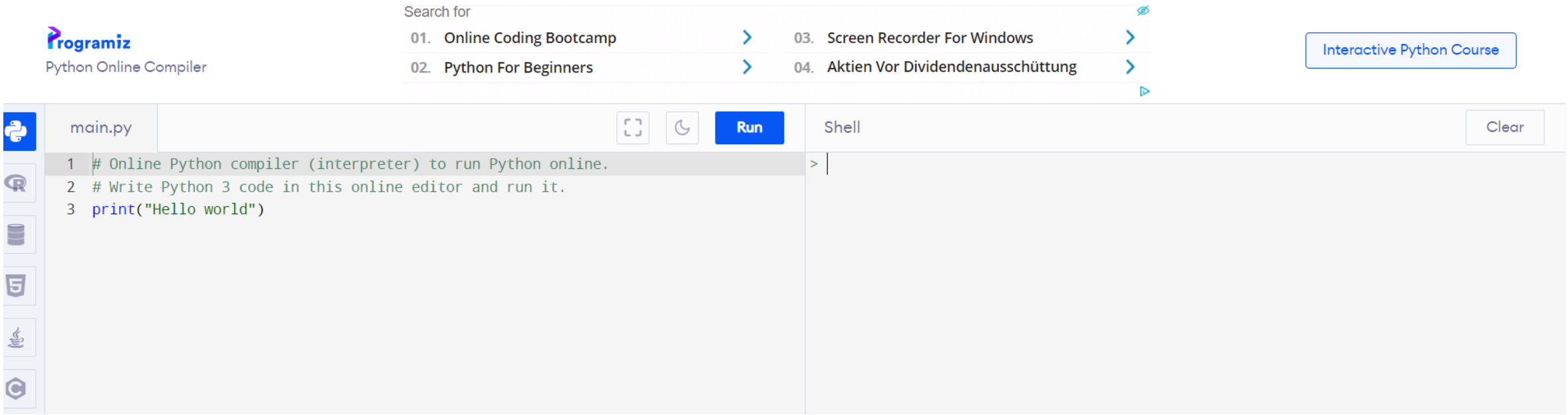


Once you finish the installation process, you can run Python.

THE FIRST PYTHON PROGRAM

Now that we have Python up and running, we can write our first Python program. Let's create a very simple program called Hello World.

Online Compiler: https://www.programiz.com/python-programming/online-compiler/



THE FIRST PYTHON PROGRAM

Type the following code in any text editor or an IDE and save it as hello_world.py

```
print("Hello, world!")
```

Run Code »

Then, run the file. You will get the following output.

```
Hello, world!
```

INDENTATION AND COMMENTS

Most of the programming languages like C, C++, and Java use braces { } to define a block of code. Python, however, uses indentation.
A code block (body of a function, loop, etc.) starts with indentation and ends with the first un-indented line. The amount of indentation is up to you, but it must be consistent throughout that block.

Generally, four whitespaces are used for indentation and are preferred over tabs. Here is an example.

```
for i in range(1,11):
    print(i)
    if i == 5:
        break
```

Run Code »

INDENTATION AND COMMENTS

Comments are very important while writing a program.
In Python, we use the hash (#) symbol to start writing a comment and use triple quotes ‘’‘ or “”“ to comment multiple lines.

```
#This is a comment
#print out Hello
print('Hello')
```

Run Code »

```
"""This is also a
perfect example of
multi-line comments"""
```

Section 1: Data

VARIABLES

A variable is a named location used to store data in the memory. It is helpful to think of variables as a container that holds data that can be changed later in the program. For example,

```
website = "apple.com"
print(website)

# assigning a new value to website
website = "programiz.com"

print(website)
```

Here, we have created a variable named website. We have assigned the value “apple.com” to the variable. Initially, the value of website was “apple.com”. Later, it was changed to “programiz.com”.

Output

```
apple.com
programiz.com
```

DATA TYPES

Every value in Python has a datatype. There are various data types in Python. Some of the important types are listed below.

- Numbers

Integers, floating point numbers fall under Python numbers category. They are defined as int, float in Python.
We can use the type() function to know which class a variable or a value belongs to.

```
a = 5
print(a, "is of type", type(a))

a = 2.0
print(a, "is of type", type(a))
```

Output

```
5 is of type <class 'int'>
2.0 is of type <class 'float'>
```

DATA TYPES

➢ Boolean

A Boolean variable can have any of the two values: True or False.

```
a = True
b = False


print("a:", a)
print("b:", b)
```

In Python, True represents the value as 1 and False as 0.

Output

```
a: True
b: False
```

DATA TYPES

- Strings

String is sequence of Unicode characters. We can use single quotes or double quotes to represent strings.

```
s = "This is a string"
print(s)
s = '''A multiline
string'''
print(s)
```

Run Code »

Output

```
This is a string
A multiline
string
```

LIST

➢ List

Python lists are one of the most versatile data types that allow us to work with multiple elements at once.

➢ Create Python Lists

In Python, a list is created by placing elements inside square brackets [], separated by commas.

```
# list of integers
my_list = [1, 2, 3]
```

```
# empty list
my_list = []

# list with mixed data types
my_list = [1, "Hello", 3.4]
```

A list can also have another list as an item. This is called a nested list.

```
# nested list
my_list = ["mouse", [8, 4, 6], ['a']]
```

LIST

➢ Access List Elements

We can use the index operator [] to access an item in a list. In Python, indices start at 0.

```
my_list = ['p', 'r', 'o', 'b', 'e']

# first item
print(my_list[0])  # p

# third item
print(my_list[2])  # o

# fifth item
print(my_list[4])  # e
```

Output

```
p
o
e
```

LIST

➢ Access List Elements

We can use the index operator [] to access an item in a list. In Python, indices start at 0.

```
# Error!
print(my_list[5])
```

Output

```
Traceback (most recent call last):
  File "<string>", line 12, in <module>
IndexError: list index out of range
```

LIST

- Add/Change List Elements

```
# Correcting mistake values in a list
odd = [2, 4, 6, 8]

# change the 1st item
odd[0] = 1

print(odd)

# Appending and Extending lists in Python
odd = [1, 3, 5]

odd.append(7)

print(odd)
```

Output

```
[1, 4, 6, 8]
[1, 3, 5, 7]
```

DICTIONARY

➢ Dictionary

Python dictionary is an unordered collection of items. Each item of a dictionary has a key/value pair.

➢ Creating Python Dictionary

Creating a dictionary is as simple as placing items inside curly braces {} separated by commas.

```
# empty dictionary
my_dict = {}

# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}

# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}
```

DICTIONARY

➢ Accessing Elements from Dictionary

Keys can be used either inside square brackets [] or with the get() method.

```
# get vs [] for retrieving elements
my_dict = {'name': 'Jack', 'age': 26}

# Output: Jack
print(my_dict['name'])

# Output: 26
print(my_dict.get('age'))

# Trying to access keys which doesn't exist throws error
# Output None
print(my_dict.get('address'))

# KeyError
print(my_dict['address'])
```

Output

```
Jack
26
None
Traceback (most recent call last):
  File "<string>", line 15, in <module>
    print(my_dict['address'])
KeyError: 'address'
```

DICTIONARY

➢ Changing and Adding Dictionary element

```
# Changing and adding Dictionary Elements
my_dict = {'name': 'Jack', 'age': 26}

# update value
my_dict['age'] = 27

#Output: {'age': 27, 'name': 'Jack'}
print(my_dict)

# add item
my_dict['address'] = 'Downtown'

# Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
print(my_dict)
```

Output

```
{'name': 'Jack', 'age': 27}
{'name': 'Jack', 'age': 27, 'address': 'Downtown'}
```

OPERATORS

➢ Operators are special symbols in Python that carry out arithmetic or logical computation.
For example:

```
>>> 2+3
5
```

❖ Arithmetic operators

Operator	Meaning	Example
+	Add two operands or unary plus	x + y+ 2
-	Subtract right operand from the left or unary minus	x - y- 2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y

OPERATORS

❖ Comparison operators

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y

❖ Logical operators

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

Section 2: Flow Control

IF...ELSE STATEMENT

- if..else

```
# Program checks if the number is positive or negative
# And displays an appropriate message

num = 3

# Try these two variations as well.
# num = -5
# num = 0

if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

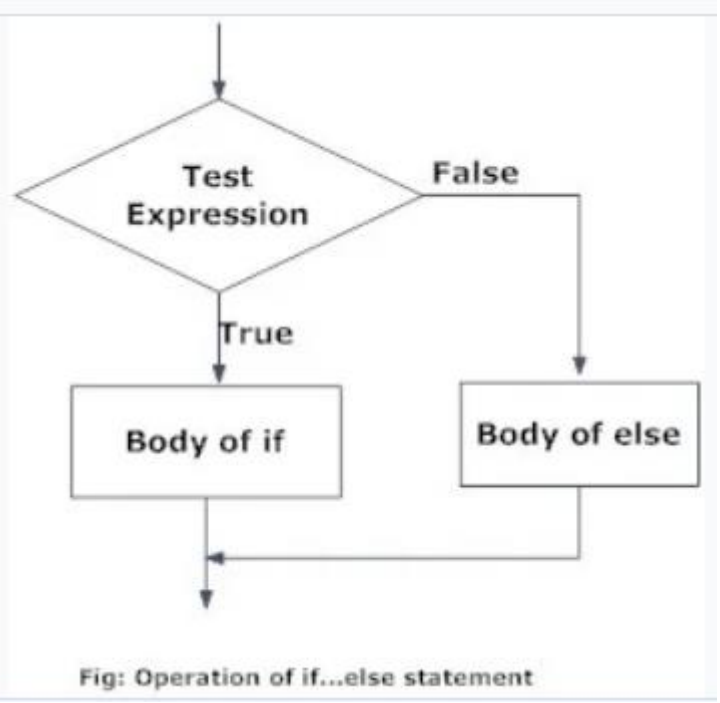


Fig: Operation of if...else statement

Output

```
Positive or Zero
```

IF...ELSE STATEMENT

- if...elif...else

```
'''In this program,
we check if the number is positive or
negative or zero and
display an appropriate message'''

num = 3.4

# Try these two variations as well:
# num = 0
# num = -4.5

if num > 0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
```

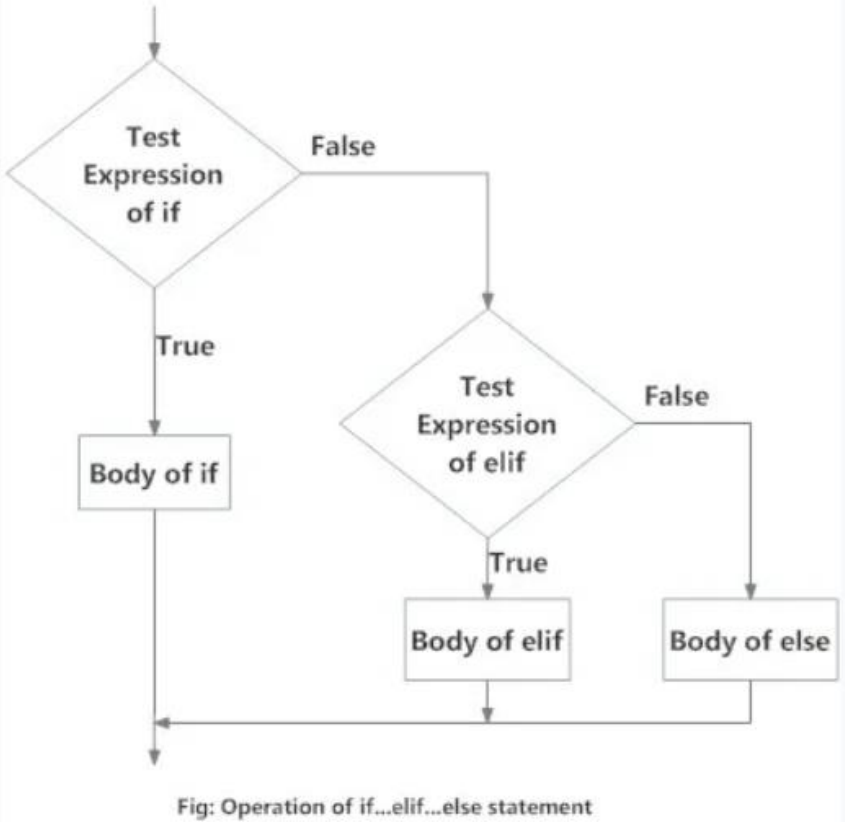


Fig: Operation of if...elif...else statement

Output

```
Positive or Zero
```

FOR LOOP

- Loop
- The for loop in Python is used to iterate over a sequence (list, string) or other iterable objects.
- We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).

```
# Program to find the sum of all numbers stored in a list

# List of numbers
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]

# variable to store the sum
sum = 0

# iterate over the list
for val in numbers:
    sum = sum+val

print("The sum is", sum)
```

Output

```
The sum is 48
```

Fig: operation of for loop

WHILE LOOP

- While
  - The while loop in Python is used to iterate over a block of code if the test expression (condition) is true.

```
# Program to add natural
# numbers up to
# sum = 1+2+3+...+n

# To take input from the user,
# n = int(input("Enter n: "))

n = 10

# initialize sum and counter
sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1    # update counter

# print the sum
print("The sum is", sum)
```

Output

```
Enter n: 10
The sum is 55
```

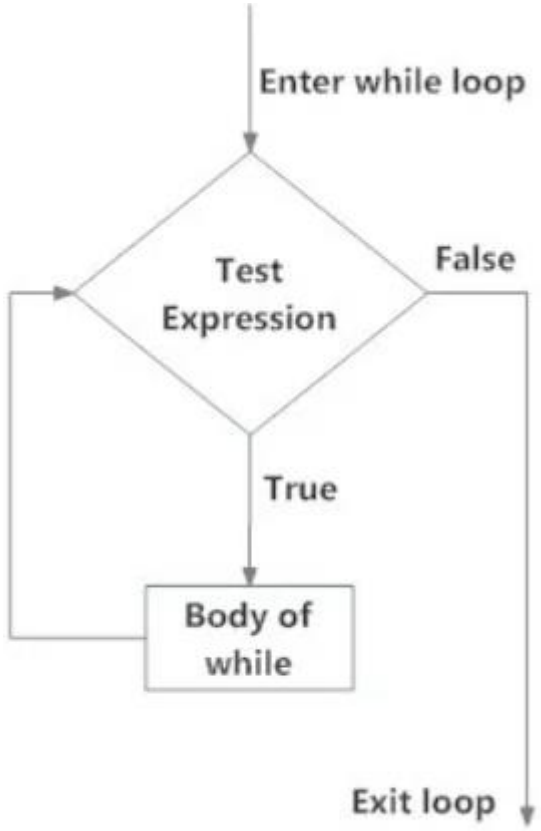


Fig: operation of while loop

BREAK AND CONTINUE

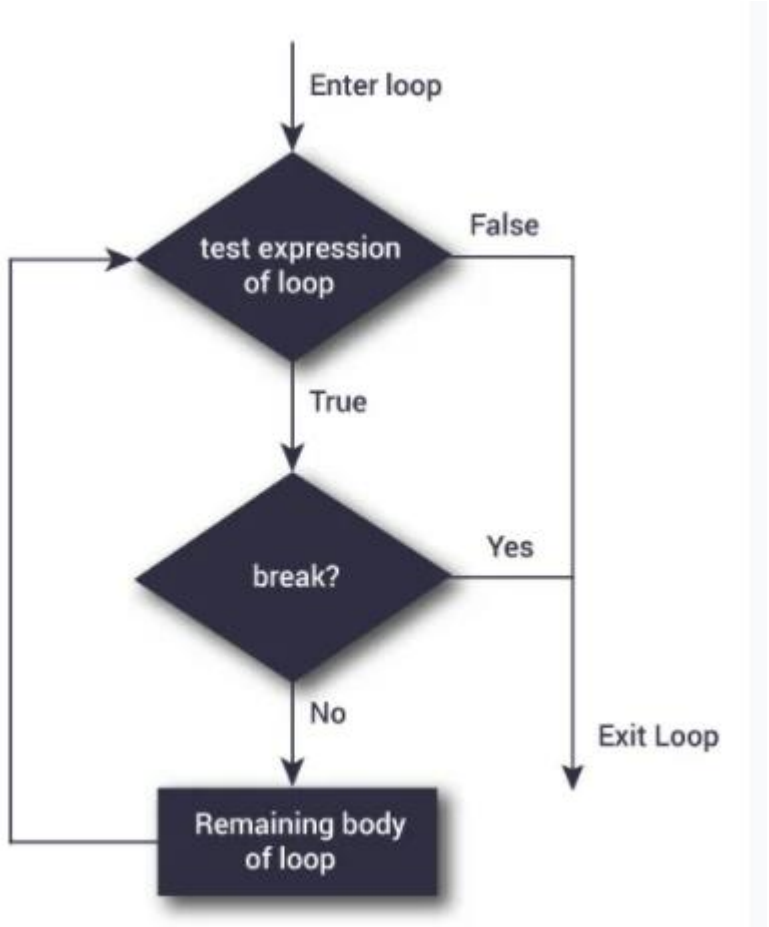
In Python, break and continue statements can alter the flow of a normal loop.

- The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

```
# Use of break statement inside the loop

for val in "string":
    if val == "i":
        break
    print(val)

print("The end")
```

Output

```
s
t
r
The end
```



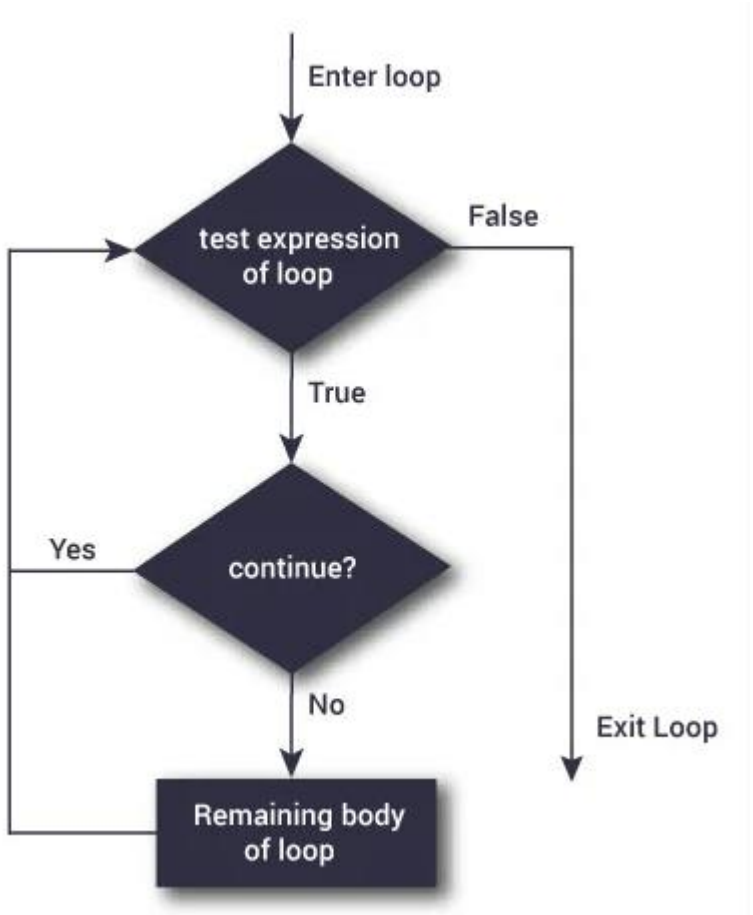
BREAK AND CONTINUE

- The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

```
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
        continue
    print(val)

print("The end")
```

Output

```
s
t
r
n
g
The end
```



LOOP WITH ELSE

For and While loop can have an optional else block as well. The else part is executed if no break occurs when loop is completed.

```
digits = [0, 1, 5]

for i in digits:
    print(i)
else:
    print("No items left.")
```

Output

```
0
1
5
No items left.
```

```
'''Example to illustrate
the use of else statement
with the while loop'''

counter = 0

while counter < 3:
    print("Inside loop")
    counter = counter + 1
else:
    print("Inside else")
```

Output

```
Inside loop
Inside loop
Inside loop
Inside else
```

Section 3: Functions and Modules

FUNCTIONS

- In Python, a function is a group of related statements that performs a specific task.
  - ✓ Functions help break our program into smaller and modular chunks. As program grows larger and larger, functions make it more organized and manageable.
  - ✓ Furthermore, it avoids repetition and makes the code reusable.
  - ✓ The return statement is used to exit a function and go back to the place from where it was called.

```
def absolute_value(num):
    """This function returns the absolute
    value of the entered number"""

    if num >= 0:
        return num
    else:
        return -num


print(absolute_value(2))

print(absolute_value(-4))
```

Output

```
2
4
```

IMPORT

- A module is a file containing Python definitions and statements. Python modules have a filename and end with the extension .py.
- Definitions inside a module can be imported to another module in Python. We use the import keyword to do this.

For example, we can import the math module by typing the following line:

```
import math
```

We can use the module in the following ways:

```
print(math.pi)
```

Output

```
3.141592653589793
```

Now all the definitions inside math module are available in our scope. We can also import some specific attributes and functions only, using the from keyword. For example:

```
>>> from math import pi
>>> pi
3.141592653589793
```

OBJECTS AND CLASSES

- Python is an object-oriented programming language.
- An object is simply a collection of data (variables) and methods (functions) that act on those data. Similarly, a class is a blueprint for that object.

❖ **Defining a Class in Python**

➢ Like function definitions begin with the def keyword in Python, class definitions begin with a class keyword.

```
class Person:
    "This is a person class"
    age = 10

    def greet(self):
        print('Hello')


# Output: 10
print(Person.age)

# Output: <function Person.greet>
print(Person.greet)

# Output: "This is a person class"
print(Person.__doc__)
```

OBJECTS AND CLASSES

❖ **Creating an Object in Python**

➢ The procedure to create an object is like a function call.

```
>>> harry = Person()
```

```
class Person:
    "This is a person class"
    age = 10

    def greet(self):
        print('Hello')


# create a new object of Person class
harry = Person()

# Output: <function Person.greet>
print(Person.greet)

# Output: <bound method Person.greet of <__main__.Person object>>
print(harry.greet)

# Calling object's greet() method
# Output: Hello
harry.greet()
```

Section 4: Advanced Topics

DECORATORS

Python has an interesting feature called decorators to add functionality to an existing code.

- We had functions that took in parameters like:

```
def divide(a, b):
    return a/b
```

This function has two parameters, a and b. We know it will give an error if we pass in b as 0.

```
>>> divide(2,5)
0.4
>>> divide(2,0)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
```

DECORATORS

If there is an existing function to check for this case that will cause the error.

```
def smart_divide(func):
    def inner(a, b):
        print("I am going to divide", a, "and", b)
        if b == 0:
            print("Whoops! cannot divide")
            return

        return func(a, b)
    return inner
```

We can use the @ symbol along with the name of the decorator function and place it above the definition of the function to be decorated. For example,

```
@smart_divide
def divide(a, b):
    print(a/b)
```

DECORATORS

Before using decorators, divide function doesn’t define error message

```
>>> divide(2,5)
0.4
>>> divide(2,0)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
```

This new implementation will return message if the error condition arises.

```
>>> divide(2,5)
I am going to divide 2 and 5
0.4

>>> divide(2,0)
I am going to divide 2 and 0
Whoops! cannot divide
```

This is just a syntactic sugar to implement decorators.

EXCEPTION HANDLING

When exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled. If never handled, an error message is displayed, and our program comes to a sudden unexpected halt.

```
def divide(a, b):
    return a/b
```

Output

```
>>> divide(2,5)
0.4
>>> divide(2,0)
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
```

EXCEPTION HANDLING

In Python, exceptions can be handled using a try statement.

The critical operation which can raise an exception is placed inside the try clause. The code that handles the exceptions is written in the except clause.

We can choose what operations to perform once we have caught the exception. Here is a simple example.

```
# import module sys to get the type of exception
import sys

randomList = ['a', 0, 2]

for entry in randomList:
    try:
        print("The entry is", entry)
        r = 1/int(entry)
        break
    except:
        print("Oops!", sys.exc_info()[0], "occurred.")
        print("Next entry.")
        print()
print("The reciprocal of", entry, "is", r)
```

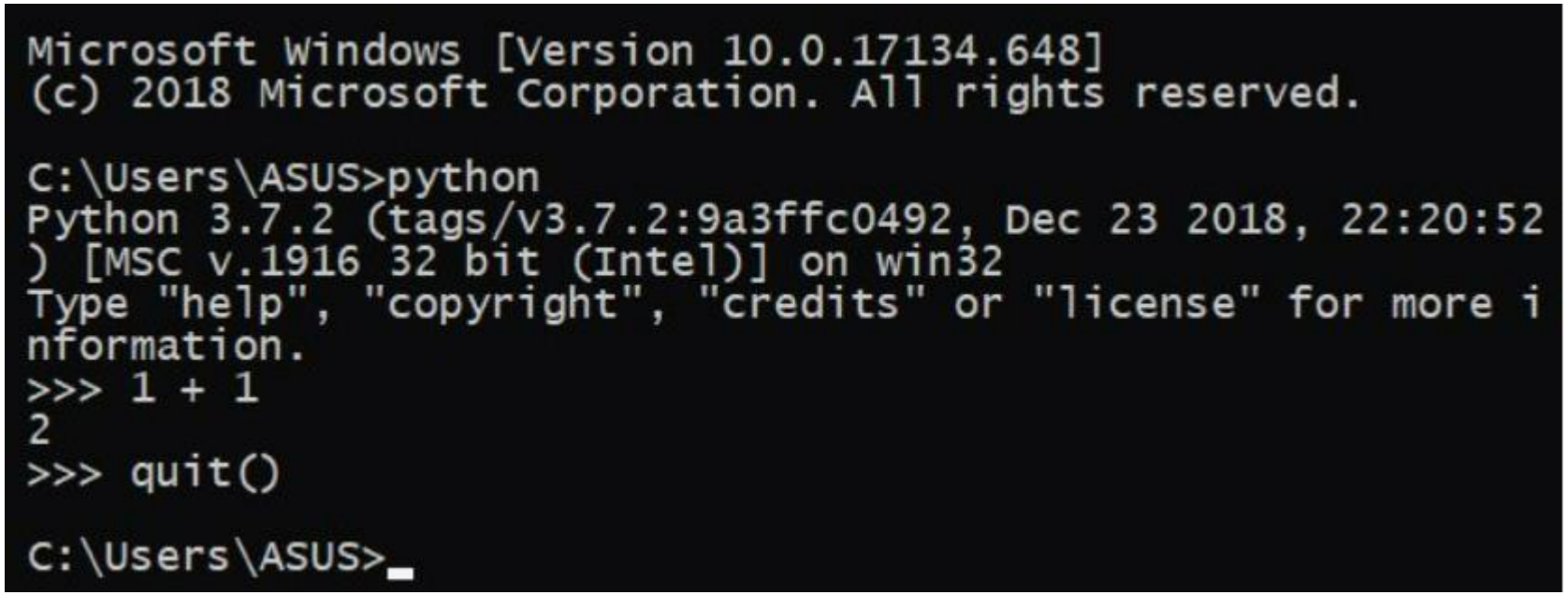
Output

```
The entry is a
Oops! <class 'ValueError'> occurred.
Next entry.

The entry is 0
Oops! <class 'ZeroDivisionError'> occured.
Next entry.

The entry is 2
The reciprocal of 2 is 0.5
```

Part 3: How to run Python

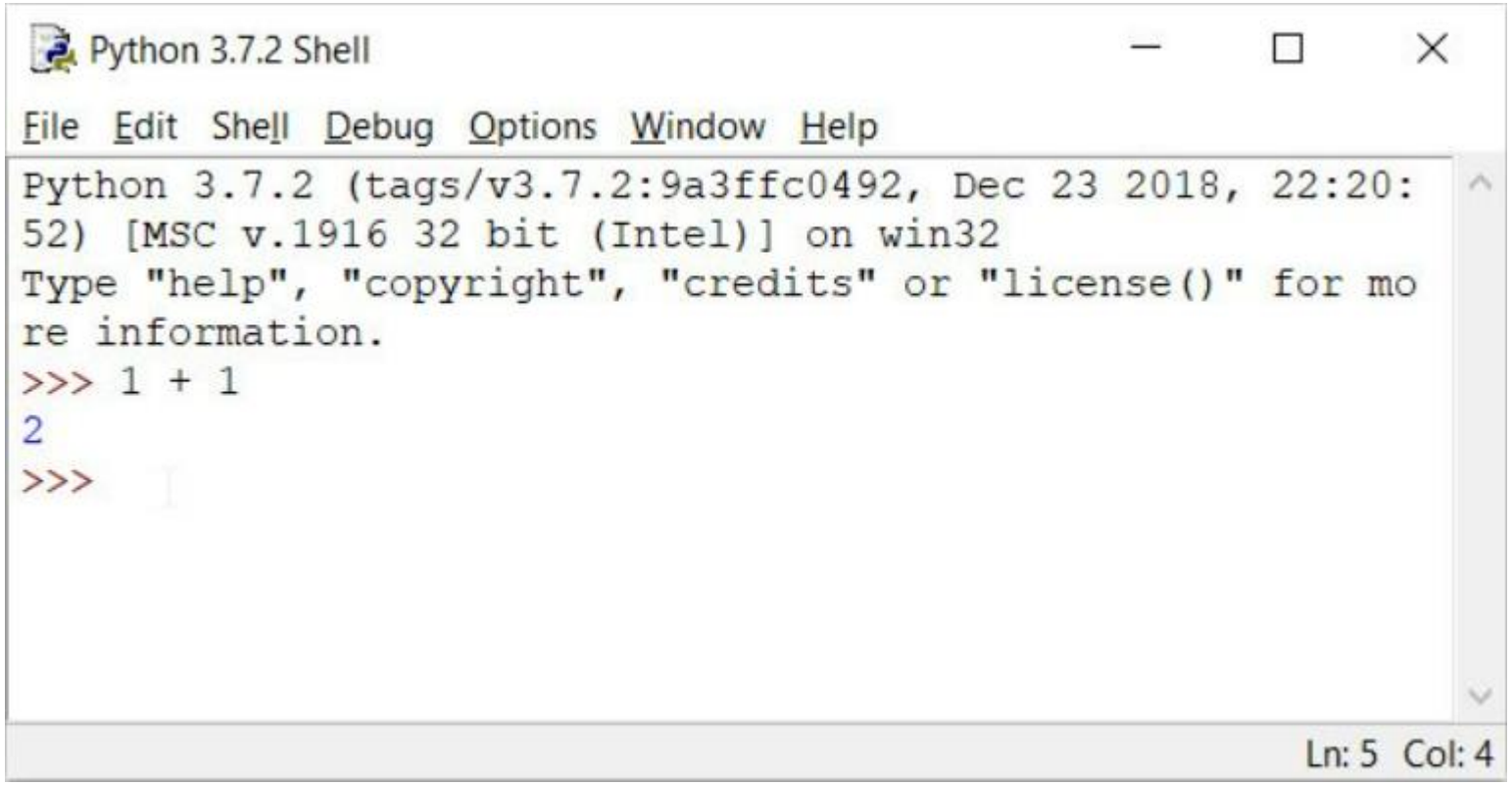
RUN PYTHON IN IMMEDIATE MODE

Once Python is installed, typing python in the command line will invoke the interpreter in immediate mode. We can directly type in Python code, and press Enter to get the output.



RUN PYTHON IN THE INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

We can use any text editing software to write a Python script file.
When you install Python, an IDE named IDLE is also installed. You can use it to run Python on your computer. It's a decent IDE for beginners.



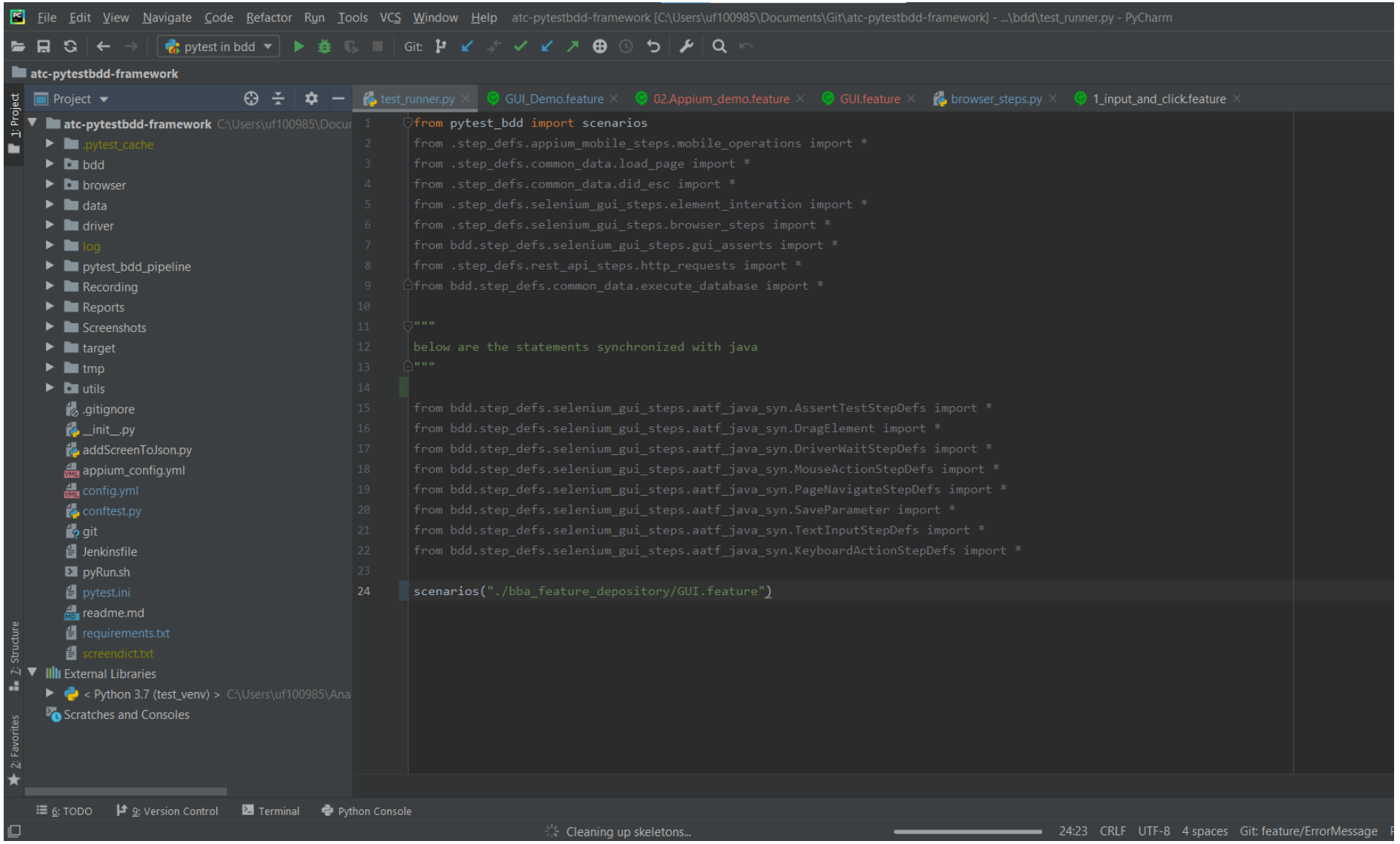
PYCHARM

Pycharm community is the most popular python free IDE

https://www.jetbrains.com/pycharm/

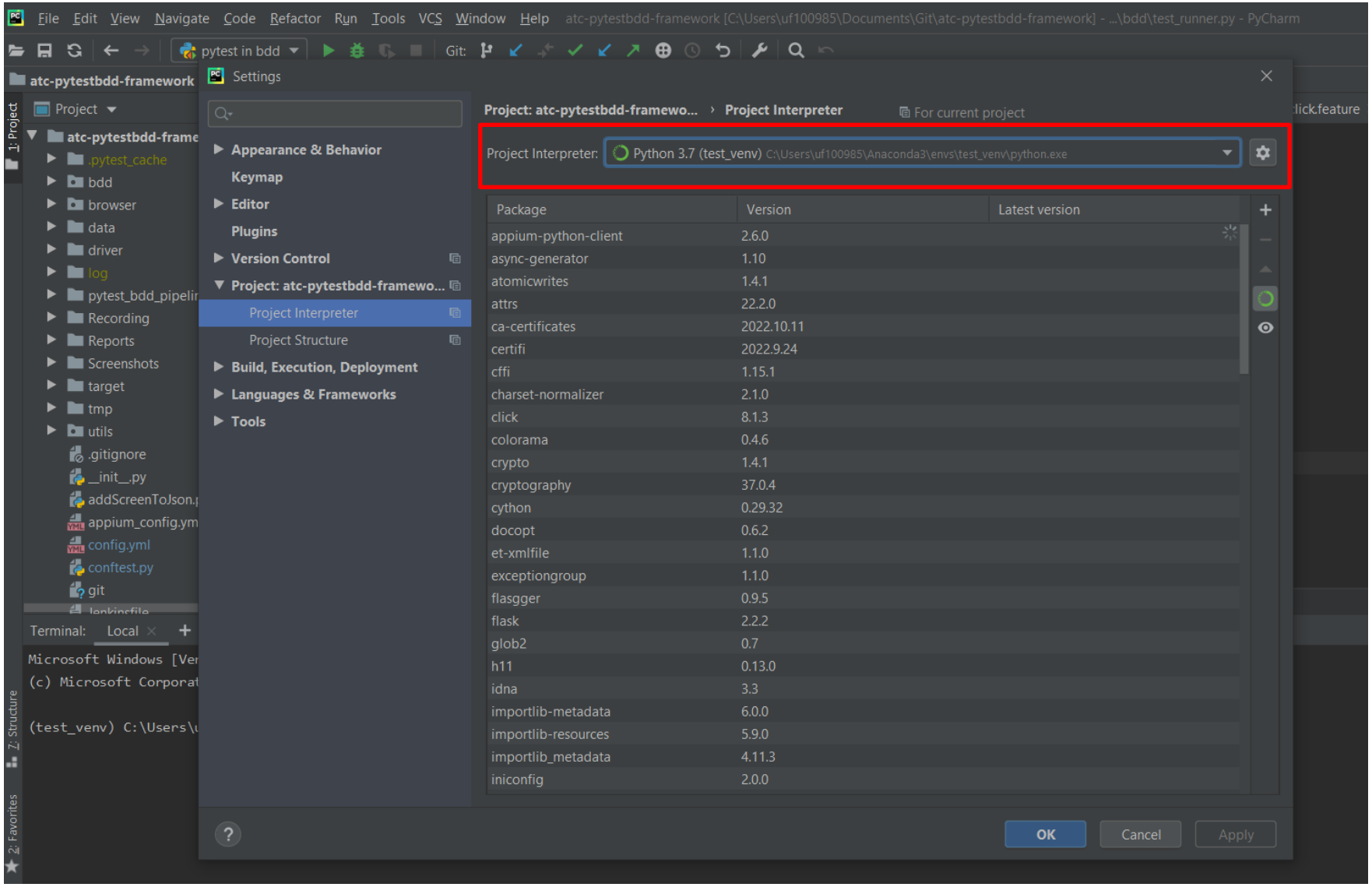
PYCHARM

Open our program as a project in PyCharm, we will get it:



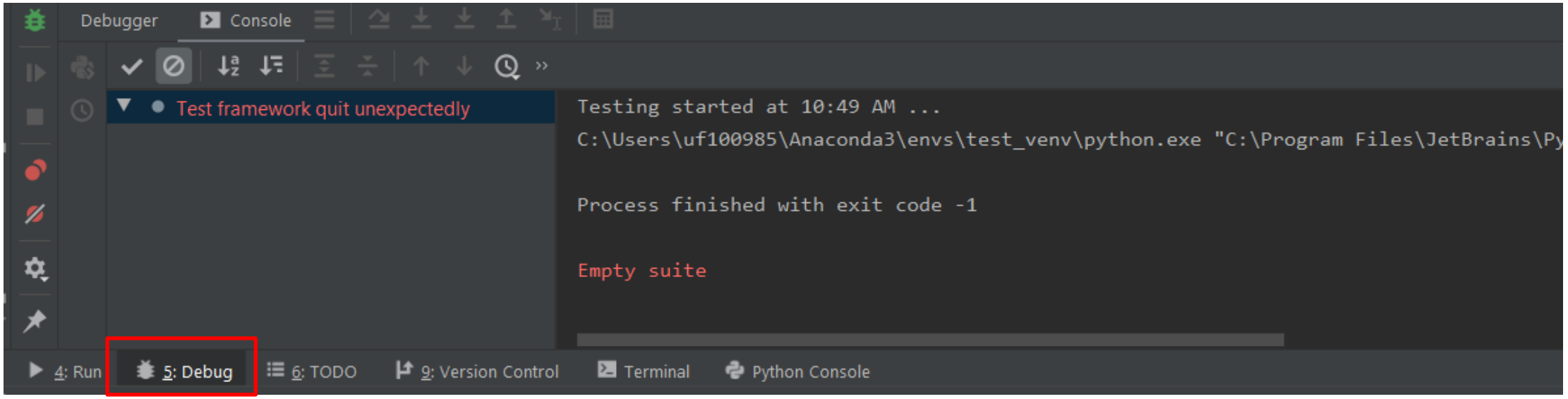
PYCHARM

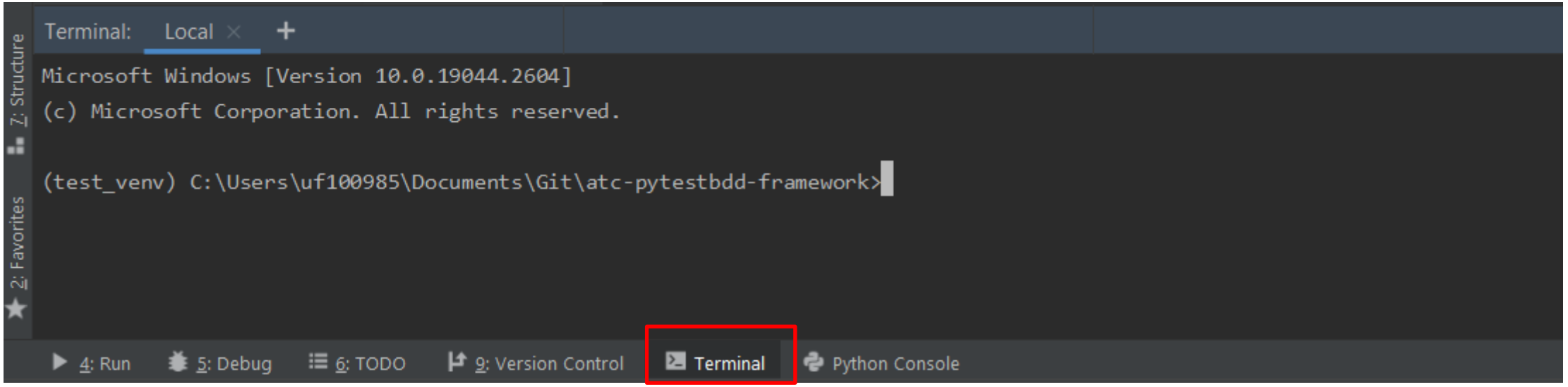
Select Python Install Location to set interpreter



PYCHARM

We can debug our program in Debug window or run program directly in Run window or Terminal





Thank you!